

ENHANCE YOUR BASH SCRIPTS

DYNAMIC TERMINAL CONTROL AND SIGNAL HANDLING



Written By: Hosni Zaaraoui

Introduction:

Bash scripting becomes a whole new experience when combining **tput** commands and **trap** for dynamic terminal management and graceful script handling.

With **tput**, you can dynamically style and control your terminal scripts, and with **trap**, you can ensure they are resilient and responsive to signals. Combining both tools elevates your scripts to the next level, making them visually appealing and user-friendly.

Start experimenting with these commands, and watch how your scripts come to life!

Understanding tput Commands

The **tput** utility allows you to control terminal features in a portable way. It interacts with terminfo capabilities to make your scripts more dynamic and visually appealing.

cput(1)

General Commands Manual

tput(1)

```
NAME
    tput, reset - initialize a terminal or query terminfo database

SYNOPSIS
    tput [-Ttype] capname [parameters]
    tput [-Ttype] [-x] clear
    tput [-Ttype] init
    tput [-Ttype] reset
    tput [-Ttype] longname
    tput -S <<
    tput -V

DESCRIPTION
    The tput utility uses the terminfo database to make the values of terminal-dependent capabilities and information available to the shell (see sh(1)), to initialize or reset the terminal, or return the long name of the requested terminal type. The result depends upon the capability's type:

    string
        tput writes the string to the standard output. No trailing newline is supplied.

    integer
        tput writes the decimal value to the standard output, with a trailing newline.

    boolean
        tput simply sets the exit code (0 for TRUE if the terminal has the capability, 1 for FALSE if it does not), and writes nothing to the standard output.

    Before using a value returned on the standard output, the application should test the exit code (e.g., $?, see sh(1)) to be sure it is 0. (See the EXIT CODES and DIAGNOSTICS sections.) For a complete list of capabilities and the capname associated with each, see terminfo(5).
```

Commonly Used tput Commands

1. Hiding and Showing the Cursor

Command:

`tput civis` (hide cursor) and `tput cnorm` (restore cursor)

Example:

```
echo "Processing... Please wait"
tput civis  # Hide the cursor
sleep 5
tput cnorm  # Restore the cursor
echo "Done!"
```

2. Moving the Cursor

Command:

`tput cup [row] [col]`

Example:

```
tput clear
tput cup 5 10  # Move the cursor to row 5, column 10
echo "Hello, world!"
```

3. Clearing Sections of the Screen

Command:

`tput el` (clear to the end of the line) or `tput ed` (clear to the end of the screen)

Example:

```
echo -n "This will be cleared!"  
sleep 2  
tput el  # Clears the line
```

4. Text Styling

Command:

`tput bold`: Enable bold text
`tput sgr0`: Reset text attributes

Example:

```
echo "$(tput bold)This is bold text$(tput sgr0)"
```

5. Changing Colors

Command:

`tput setaf [color]` (foreground), `tput setab [color]` (background)

Example:

```
tput setaf 4  # Set text to blue  
echo "Blue text"  
tput sgr0    # Reset attributes
```

Combined Example: A Simple Progress Bar

```
tput civis
tput clear
for i in $(seq 1 20); do
    echo -ne "$(tput cup 5 10)["
    for j in $(seq 1 $i); do echo -ne "#"; done
    echo -ne "]" $((i * 5))% Complete"
    sleep 0.2
    tput el # Clear rest of the line
done
tput cnorm
```

Enhancing Bash with trap for Signal Handling

The **trap** command lets you catch and handle system signals, allowing your script to respond gracefully to events like interrupts or terminal resizing.

Mastering trap commands lets you handle a wide range of scenarios in your scripts, from cleaning up resources to responding to terminal signals dynamically. Combined with tput, you can build highly interactive and robust Bash scripts.

Commonly Used Signals and Examples

1. Handle Interrupts (CTRL+C)

Signal: **SIGINT**

Example:

```
trap "echo 'Interrupt detected. Exiting...'; exit 1"
SIGINT
while true; do
    echo "Running... Press CTRL+C to exit."
    sleep 2
done
```

2. Respond to Window Resizing

Signal: **SIGWINCH**

Example:

```
trap 'clear; echo "Window resized! Adjust your layout
here."' WINCH
while true;do
    sleep 1
done
```

3. Graceful Cleanup on Exit

Signal: **EXIT**

Example:

```
trap 'tput cnorm; echo "Exiting gracefully"; exit'
EXIT

tput civis

echo "Script running... Press CTRL+C to quit."

sleep 5
```

4. Terminal Stop, CTRL+Z

Signal: **SIGTSTP**

Example:

```
trap 'echo "Process stopped with CTRL+Z!"' SIGTSTP

while true; do
    echo "Running... Press CTRL+Z to test the trap."
    sleep 2
done
```

5. Others

Signal: **SIGHUP** (Hangup)

Triggered when the terminal is disconnected or a session is closed.

Signal: SIGTERM (Termination)

Used to terminate a process gracefully (e.g., when stopping a script with kill).

Signal: SIGQUIT (Quit)

Triggered with CTRL+\ or equivalent to force a quit.

Signal: SIGCONT (Continue)

Triggered when a stopped process (e.g., via CTRL+Z) is resumed with fg or bg.

Signal: SIGQUIT (Quit)

Triggered with CTRL+\ or equivalent to force a quit.

Signal: SIGALRM (Alarm)

Triggered by a timer set using alarm.

Example:

```
trap 'echo "Timer expired!"; exit' SIGALRM
echo "Setting a 5-second alarm..."
sleep 5 &
wait
```

Combining tput and trap for Advanced Scripts

Example: Dynamic Terminal with Cleanup and Resizing Support

```
trap 'tput cnorm; echo "Exiting..."; exit' EXIT # Cleanup
on exit

trap 'clear; echo "Terminal resized!"' WINCH    # Respond
to resizing
```

```
tput civis
```

```
clear
```

```
echo "Dynamic Bash Script with tput and trap"
```

```
for i in {1..10}; do
```

```
    tput cup $i 5
```

```
    echo "Processing $i/10"
```

```
    sleep 1
```

```
done
```